P2 - 2021 Exam: Registration page (username + password validation)

25 marks (Python part of the 2021 paper). tkinter is no longer assessed, but the database + validation logic is exactly the kind of thing the mock can ask in a console program. Files: `Solutions_py/2021_registration_users_solution.py` *(window version) and* `2021_registration_console.py` *(terminal version, the one to practise).*

1. What the paper asked

Task	Requirement	Marks
1a	Show the registration screen	4
1a	**Save** stores username + password in a database called **Users**	4
1a	**Clear** empties all windows	2
1a	**Display** shows all usernames and passwords from the database	3
1b	No duplicate usernames; show a message if it already exists	5
1b	Password: minimum 8 characters **and** contains a number; otherwise reject with a message	6

Sample result to reproduce: after saving four users, Display lists `4 mire@hotmail.com P@1ssword`, `3 Robert password23`, `2 Cristina password23`, `1 Mireilla password1` (newest first, with the row id in front).

2. The idea in one analogy

A **club membership desk**. The clerk (cursor) keeps a card index (the table). Before writing a new card the clerk (1) checks the index for the same name (duplicate check), (2) checks the chosen password meets the club rules. Only then is the card filed (`commit`).

The order of checks matters for messages, not for marks: empty -> duplicate -> weak password -> save.

3. The terminal version, explained line by line (practise this one)

```
import sqlite3

conn = sqlite3.connect("Users.db")
cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS Users (
                      id       INTEGER PRIMARY KEY AUTOINCREMENT,
                      username TEXT NOT NULL,
                      password TEXT NOT NULL)""")
conn.commit()
```

- `sqlite3.connect("Users.db")` : open (or create) the database file named **Users** (the paper's name; `.db` is just the file extension).
- `id INTEGER PRIMARY KEY AUTOINCREMENT`: a number the database hands out itself (1, 2, 3, ...). It is why Display shows the numbers in front.
- `NOT NULL`: the cell may never be left empty.

```
def username_exists(username):
    cursor.execute("SELECT id FROM Users WHERE LOWER(username) = LOWER(?)",
                   (username,))
    return cursor.fetchone() is not None


def password_is_valid(password):
    # at least 8 characters AND at least one digit
    return len(password) >= 8 and any(ch.isdigit() for ch in password)
```

- `LOWER(username) = LOWER(?)`: compares both sides in lower case, so `Mireilla` and `mireilla` count as the **same** user (the 2021 screenshot types a lower-case name to trigger the message).
- `fetchone()` returns a row (a tuple) or `None`. `is not None` = "something was found" = the name exists. Analogy: the clerk comes back from the index with a card or with empty hands.
- `len(password) >= 8`: criterion (a). `any(ch.isdigit() for ch in password)`: criterion (b). Read it as "is there **any** character in the password that is a digit?" Walks the string letter by letter and stops as soon as one is a number.
- `and`: the paper says rejecting unless **both** are met.

```
def register():
    username = input("Enter your username: ").strip()
    password = input("Enter your password: ")
    if username == "" or password == "":
        print("Username and password cannot be empty.")
    elif username_exists(username):
        print("Username already exists – choose another one.")
    elif not password_is_valid(password):
        print("The password requirements are not met "
              "(minimum 8 characters and at least one number).")
    else:
        cursor.execute("INSERT INTO Users (username, password) "
                       "VALUES (?, ?)", (username, password))
        conn.commit()
        print("Saved.")
```

- Chain `if / elif / elif / else`: only the **first** failing check runs, then Python jumps to the end. The `else` (everything fine) saves.
- `.strip()` removes accidental spaces at the ends of the username.
- `INSERT ... VALUES (?, ?)` with `(username, password)`: fill-in-the-blank form (safe).
- `conn.commit()`: file the card permanently.

```
def display_all():
    cursor.execute("SELECT id, username, password FROM Users ORDER BY id")
    rows = cursor.fetchall()
    if not rows:
        print("(no users saved yet)")
    for user_id, username, password in rows:
        print(user_id, username, password)


def main():
    choice = ""
    while choice != "3":
        print("\n1) Register   2) Display all   3) Quit")
        choice = input("Choose: ").strip()
        if choice == "1":
            register()
        elif choice == "2":
            display_all()
        elif choice != "3":
            print("Please type 1, 2 or 3.")
    conn.close()
```

- `for user_id, username, password in rows:`: each row is a 3-item tuple; Python unpacks it into three names.
- `main()` loop: `while choice != "3":` keeps showing the menu until the user types 3. `if __name__ == "__main__":` means "only start the menu when this file is run directly".

4. If the examiner makes it a window again (tkinter map)

Console idea	tkinter equivalent in `2021_registration_users_solution.py`
`input("username")`	`username_entry.get()`
`print(...)` of a warning	`messagebox.showwarning(title, message)`

Console idea	tkinter equivalent in `2021_registration_users_solution.py`
clear screen	`entry.delete(0, tk.END)` ; Text widget: `box.delete("1.0", tk.END)`
print rows	`display_box.insert(tk.END, text)`
menu choice	`tk.Button(root, text="Save", command=save_user)` (no brackets after the function name!)

The validation functions `username_exists` and `password_is_valid` are **identical** in both versions: that is the point; the GUI is only a coat of paint.

5. Traps

- **TRAP:** `command=save_user()` runs the function immediately when the window is built. Write `command=save_user`.
- **TRAP:** forgetting `conn.commit()`: the user "saved" but Display shows nothing after restarting.
- **TRAP:** checking the password rule with `or` (accepts weak passwords). Need `and`.
- **TRAP:** `password.isdigit()` only tests if the **whole** string is digits. You need "contains a digit": `any(ch.isdigit() ...)`.
- **TRAP:** `sqlite3.OperationalError: no such table: Users` means you queried before `CREATE TABLE` ran (or opened a different `.db` file in another folder).

6. Anticipated questions

1. *Why store passwords in plain text? Is it safe?* (discussion, 2 marks) No: real systems store a **hash**, e.g. `hashlib.sha256(password.encode()).hexdigest()`, and compare hashes.
2. *Add a rule: password must contain an upper-case letter.* `has_upper = any(ch.isupper() for ch in password)` and add `and has_upper`.
3. *Make the username check case-sensitive.* Remove the `LOWER()` calls.
4. *Stop duplicates in the database itself.* `username TEXT UNIQUE NOT NULL`; catch `sqlite3.IntegrityError` when inserting.
5. *Display in order of registration.* `ORDER BY id` (ascending); the paper's screenshot is descending (`ORDER BY id DESC`).
6. *Limit the password to at most 20 characters.* `8 <= len(password) <= 20`.
7. *Add a "Delete user" function.* `DELETE FROM Users WHERE username = ?` then `commit()`.
8. *What does `AUTOINCREMENT` do?* Database numbers each new row automatically.

7. Quick-reference box

```
exists?   SELECT id FROM T WHERE LOWER(col)=LOWER(?)   -> fetchone() is not None
pw rule   len(pw) >= 8 and any(c.isdigit() for c in pw)
save      INSERT INTO T (a, b) VALUES (?, ?); conn.commit()
show      SELECT ... ORDER BY id; for a, b, c in cur.fetchall(): print(...)
clear UI  entry.delete(0, tk.END)      text.delete("1.0", tk.END)
```